

2025 年北京市大学生集成电路设计大赛

题目三 含通用寄存器的 8 位多功能总线处理器

命题老师：北京石油化工学院 田小平 北京石油化工学院学生不得选取

这是一种含通用寄存器的 8 位多功能总线处理器，具有 R0 - R3 四个通用寄存器，经总线连接。控制电路统一管控，能通过运算器执行加减计算，还可进行寄存器载数、寄存器间转存，以灵活高效的数据处理应对多样任务。所有电路的设计均不可使用 IP 核进行设计。题目不涉及实体硬件电路，所有功能的设计及仿真均在 EDA 开发环境中实现，竞赛根据总体设计框图及说明、各个模块电路设计说明、时序说明、仿真结果、资源报告、设计总结和程序源代码评定成绩。

一、设计任务

设计一个含通用寄存器的 8 位多功能总线处理器，图 1 为其总框图，由总线三态输入模块、寄存器控制模块、运算部分模块和控制电路模块构成。总线三态输入模块负责实现外部数据或寄存器数据向总线输送数据；寄存器控制模块负责将总线中的数据暂时存储到对应的寄存器；运算部分模块负责将暂存在 A 寄存器中的数据与总线中的数据，按照指定的加减法选择项进行运算并存储到 G 寄存器；控制电路模块则负责配合功能需求，按特定的节拍协调寄存器、运算器和总线控制所需的信号时序。

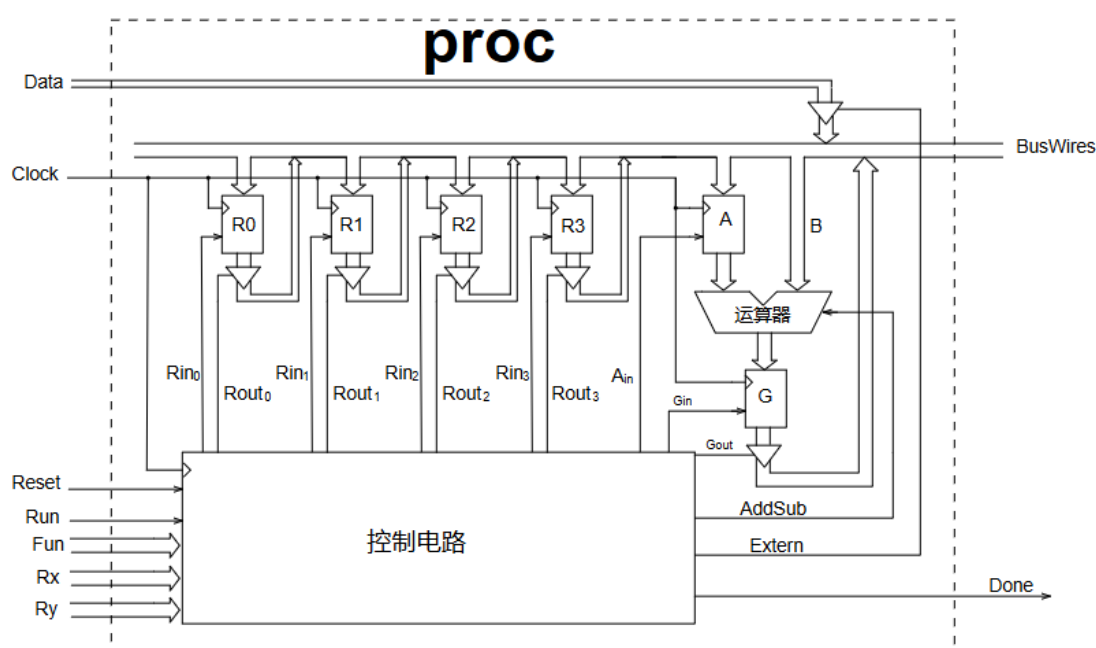


图 1 总线处理器总框图

该处理器的输入输出信号可参考表 1 所示。

表 1 为输入输出含义表

变量名	含义	变量名	含义
Data	外部输入单字节数据	Rx	发送寄存器选择
Clock	工作时钟	Ry	接收寄存器选择
Reset	全局复位端	BusWires	数据交换总线
Run	启动工作	Done	处理状态指示
Fun	功能设置		

该通用寄存器的 8 位多功能总线处理器内部的 R0-R3, A 和 G 共 6 个寄存器，需要设计 4 种运算功能，分别是载数，转存，加法和减法运算。该通用寄存器的功能描述见表 2 所示。

表 2 为功能操作表

操作	功能编码	执行的功能	含义
Load Rx, Data	00	$Rx \leftarrow Data$	将外部数据送到指定的 Rx 寄存器
Move Rx, Ry	01	$Rx \leftarrow [Ry]$	将 Ry 中的数据转存到 Rx 寄存器
ADD Rx, Ry	10	$Rx \leftarrow [Rx] + [Ry]$	将 Rx 和 Ry 中的数据相加后送到 Rx 寄存器
SUB Rx, Ry	11	$Rx \leftarrow [Rx] - [Ry]$	将 Rx 和 Ry 中的数据相减后送到 Rx 寄存器

二、要求

(1) 仿真器件选择 Cyclone EP4CE55F23C8，所有输入输出信号均保留仿真接口。

(2) 总线三态输入模块

实现外部数据或寄存器数据向总线输送数据。当三态控制信号为 1 时，数据可送到总线上；当三态控制信号为 0 时，数据不能送到总线上。

(3) 寄存器控制模块

处理器内部的 R0-R3 共 4 个寄存器能正常地存储或输出数据。

(4) 运算部分模块

处理器内部的 A 寄存器和 G 寄存器能配合 R0-R3 完成正常的加减法运算。

(5) 控制电路模块

根据功能需求，通过节拍控制产生对应的控制信号。

三、设计报告内容

- (1) 设计思路概述；
- (2) 总体设计框图及详细说明；
- (3) 时序说明：给出必要的理论分析与计算；
- (4) 模块设计框图、引脚说明、相关时序；
- (5) 程序代码及必要的注释；
- (6) 结果：给出系统编译综合结果；按照给定的测试文件仿真结果并给出适当分析；
- (7) 结论；
- (8) 其它需要说明的内容。

五、评分标准

	项目	主要内容	满分
设计报告	系统方案	设计思路，总体设计框图及说明，必要的理论分析与计算	5
	电路与程序设计	模块设计框图，引脚说明及时序，程序及注释	10
	测试结果	编译综合结果和仿真结果完整性，结果说明与分析	10
	设计报告结构及规范性	正文结构规范，表的完整与准确性	5
	小计		30
基本设计任务	总线三态输入模块设计		5
	寄存器控制模块设计		10
	运算部分模块设计		15
	控制电路模块设计		10
发挥任务	转存功能实现		12
	减法功能实现		18
	小计		70
合计			100

Verilog 版测试激励文件 proc_tb.v:

```
`timescale 1ns/1ps
```

```
// 定义测试模块
```

```
module proc_tb();
```

```
    parameter n = 8;
```

```
    reg [7:0] Data;
```

```
    reg Reset;
```

```
    reg Run;
```

```
    reg Clock;
```

```
    reg [1:0] Fun;
```

```
    reg [1:0] Rx;
```

```
    reg [1:0] Ry;
```

```
    wire [7:0] BusWires;
```

```
    wire Done;
```

```
// 实例化被测试的 proc 模块
```

```
proc dut (
```

```
    .Data(Data),
```

```
    .Reset(Reset),
```

```
    .Run(Run),
```

```
    .Clock(Clock),
```

```
    .Fun(Fun),
```

```
    .Rx(Rx),
```

```
    .Ry(Ry),
```

```
    .Done(Done),
```

```
    .BusWires(BusWires)
```

```
);
```

```
// 时钟生成逻辑，产生周期为 10ns 的时钟信号  
always #5 Clock = ~Clock;
```

```
// 测试激励产生部分
```

```
initial begin
```

```
    // 初始化信号
```

```
    Data = 0;
```

```
    Reset = 1;
```

```
    Run = 0;
```

```
    Clock = 0;
```

```
    Fun = 0;
```

```
    Rx = 0;
```

```
    Ry = 0;
```

```
// 复位操作，保持复位高电平一段时间
```

```
#10;
```

```
Reset = 0;
```

```
// 第一步：将 0x33 载数送入 R0 寄存器
```

```
#10;
```

```
Data = 8'h33;
```

```
Fun = 0;
```

```
Rx = 0;
```

```
Ry = 0;
```

```
Run = 1;
```

```
#10;
```

```
Run = 0;
```

```
// 第二步：将 0x22 载数送入 R1 寄存器
```

```
#10;  
Data = 8'h22;  
Fun = 0;  
Rx = 1;  
Ry = 0;  
Run = 1;  
#10;  
Run = 0;
```

// 第三步：将 0x11 载数送入 R2 寄存器

```
#10;  
Data = 8'h11;  
Fun = 0;  
Rx = 2;  
Ry = 0;  
Run = 1;  
#10;  
Run = 0;
```

// 第四步：执行 $R0 + R1$ 运算并将结果保存 R0

```
#10;  
Fun = 2;  
Rx = 0;  
Ry = 1;  
Run = 1;  
#10;  
Run = 0;
```

// 第五步：执行 R0 转存并移入 R3

```
#30;
```

```
Fun = 1;
```

```
Rx = 3;
```

```
Ry = 0;
```

```
Run = 1;
```

```
#10;
```

```
Run = 0;
```

```
// 第六步：执行 R1 - R2 运算并将结果保存 R1
```

```
#50;
```

```
Fun = 3;
```

```
Rx = 1;
```

```
Ry = 2;
```

```
Run = 1;
```

```
#10;
```

```
Run = 0;
```

```
// 持续运行一段时间，观察输出和状态变化并结束
```

```
#40 $finish;
```

```
end
```

```
endmodule
```

VHDL 版测试激励文件 proc_tb.VHDL:

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
use ieee.numeric_std.all;
```

```
entity proc_tb is
```

```
end entity;
```

```
architecture behavior of proc_tb is
```

```
    constant n : integer := 8;
```

```
-- 声明输入信号，对应被测试模块的输入端口
```

```
signal Data : std_logic_vector(7 downto 0);
```

```
signal Reset : std_logic;
```

```
signal Run : std_logic;
```

```
signal Clock : std_logic;
```

```
signal Fun : std_logic_vector(1 downto 0);
```

```
signal Rx : std_logic_vector(1 downto 0);
```

```
signal Ry : std_logic_vector(1 downto 0);
```

```
-- 声明输出信号，对应被测试模块的输出端口
```

```
signal BusWires : std_logic_vector(7 downto 0);
```

```
signal Done : std_logic;
```

```
-- 组件声明
```

```
component proc is
```

```
    port(
```

```
        Data : in std_logic_vector(7 downto 0);
```

```
        Reset : in std_logic;
```

```
        Run : in std_logic;
```

```
        Clock : in std_logic;
```

```

        Fun : in std_logic_vector(1 downto 0);
        Rx : in std_logic_vector(1 downto 0);
        Ry : in std_logic_vector(1 downto 0);
        Done : out std_logic;
        BusWires : out std_logic_vector(7 downto 0)
    );
end component;

```

begin

-- 实例化被测试的 proc 模块

dut : proc

port map(

Data => Data,

Reset => Reset,

Run => Run,

Clock => Clock,

Fun => Fun,

Rx => Rx,

Ry => Ry,

Done => Done,

BusWires => BusWires

);

-- 时钟生成逻辑，产生周期为 10ns 的时钟信号

process

begin

Clock <= '0';

wait for 5 ns;

Clock <= '1';

wait for 5 ns;

end process;

-- 测试激励产生部分

process

begin

-- 初始化信号

Data <= (others => '0');

Reset <= '1';

Run <= '0';

Clock <= '0';

Fun <= "00";

Rx <= "00";

Ry <= "00";

-- 复位操作，保持复位高电平一段时间

wait for 10 ns;

Reset <= '0';

-- 第一步：将 0x33 载数送入 R0 寄存器

wait for 10 ns;

Data <= x"33";

Fun <= "00";

Rx <= "00";

Ry <= "00";

Run <= '1';

wait for 10 ns;

Run <= '0';

-- 第二步：将 0x22 载数送入 R1 寄存器

wait for 10 ns;

Data <= x"22";

Fun <= "00";

```
Rx <= "01";  
Ry <= "00";  
Run <= '1';  
wait for 10 ns;  
Run <= '0';
```

-- 第三步：将 0x11 载数送入 R2 寄存器

```
wait for 10 ns;  
Data <= x"11";  
Fun <= "00";  
Rx <= "02";  
Ry <= "00";  
Run <= '1';  
wait for 10 ns;  
Run <= '0';
```

-- 第四步：执行 $R0 + R1$ 运算并将结果保存 R0

```
wait for 10 ns;  
Fun <= "10";  
Rx <= "00";  
Ry <= "01";  
Run <= '1';  
wait for 10 ns;  
Run <= '0';
```

-- 第五步：执行 R0 转存并移入 R3

```
wait for 30 ns;  
Fun <= "01";  
Rx <= "03";  
Ry <= "00";  
Run <= '1';
```

```
wait for 10 ns;
```

```
Run <= '0';
```

```
-- 第六步：执行 R1 - R2 运算并将结果保存 R1
```

```
wait for 50 ns;
```

```
Fun <= "11";
```

```
Rx <= "01";
```

```
Ry <= "02";
```

```
Run <= '1';
```

```
wait for 10 ns;
```

```
Run <= '0';
```

```
-- 持续运行一段时间，观察输出和状态变化
```

```
wait for 40 ns;
```

```
end process;
```

```
end architecture;
```

